Hertie School

Natural Language Processing

Session 5: Basic Models I
Linear models: Naive Bayes, logistic regression & SVMs

GRAD-E1282 · Fall Semester 2026
Luis Fernando Ramírez Ruiz · l.ruiz@hertie-school.org

Today: from features to predictions



By the end of today you should be able to:

- Revisit CBOW, and see why Word2vec's output layer is already a linear classifier.
- Set up a text classification problem and identify whether it is binary, multiclass or multilabel.
- Say what Naive Bayes, logistic regression and SVMs each assume, optimise and output.
- Explain what logistic regression and SVMs minimise, and why all three models are linear.
- Read a confusion matrix and report results that are honest about the baseline.

Core Python tools

scikit-learn · imblearn

The lab will train Naive Bayes, logistic regression and an SVM on the same split.

Quiz: Sessions 3, 4 and 5

WHEN

Next week

Online, available after Session 6

WHERE

Moodle

FORMAT

5–10 questions

WHAT IT COVERS

Session 3

Vector-based representations

one-hot, bag of words, n-grams, TF-IDF

Session 4

Distributed representations

Embeddings

Session 5

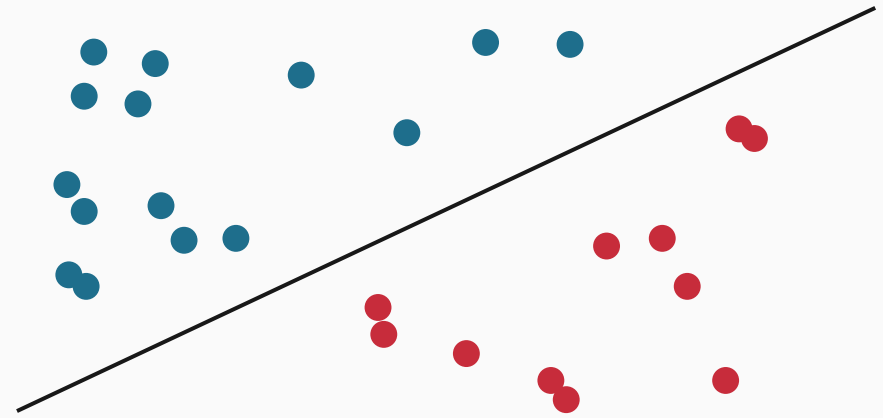
Linear classifiers (today)

Naive Bayes, logistic regression, SVM, and principles of evaluation

Counts toward your grade: the best three of four quizzes, 30% of the final grade in total.

Basic models I: linear classifiers

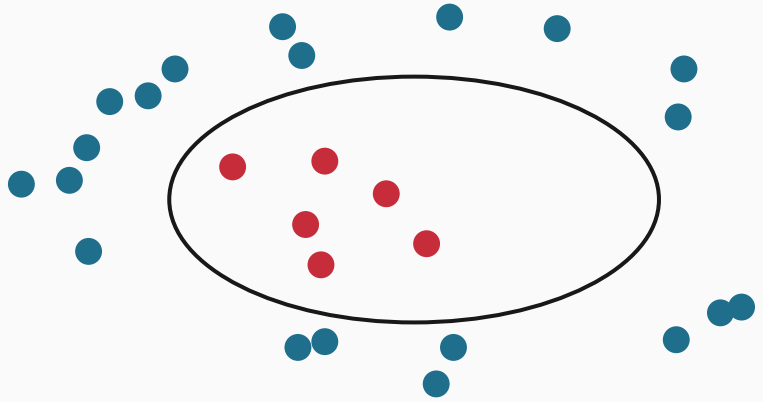
TODAY · LINEAR



Naive Bayes · logistic regression · linear SVM

The boundary $w^T x + b = 0$ is a straight line: in many dimensions, a hyperplane.

SESSION 7 · NON-LINEAR



neural networks (and today's one exception: a kernel SVM)

The boundary can bend: neural networks learn it with non-linear hidden layers.

"Basic" means linear, not weak. On sparse text with thousands of features, a straight boundary is often enough, which is why these models remain the baseline every neural model has to beat.

Session 4 in one slide: meaning learned from context

WHAT WE ASSUMED

The distributional hypothesis: words that appear in similar contexts have similar meanings. Shared neighbours, not spelling, are the only evidence Word2vec ever sees.

HOW WE LEARNED IT

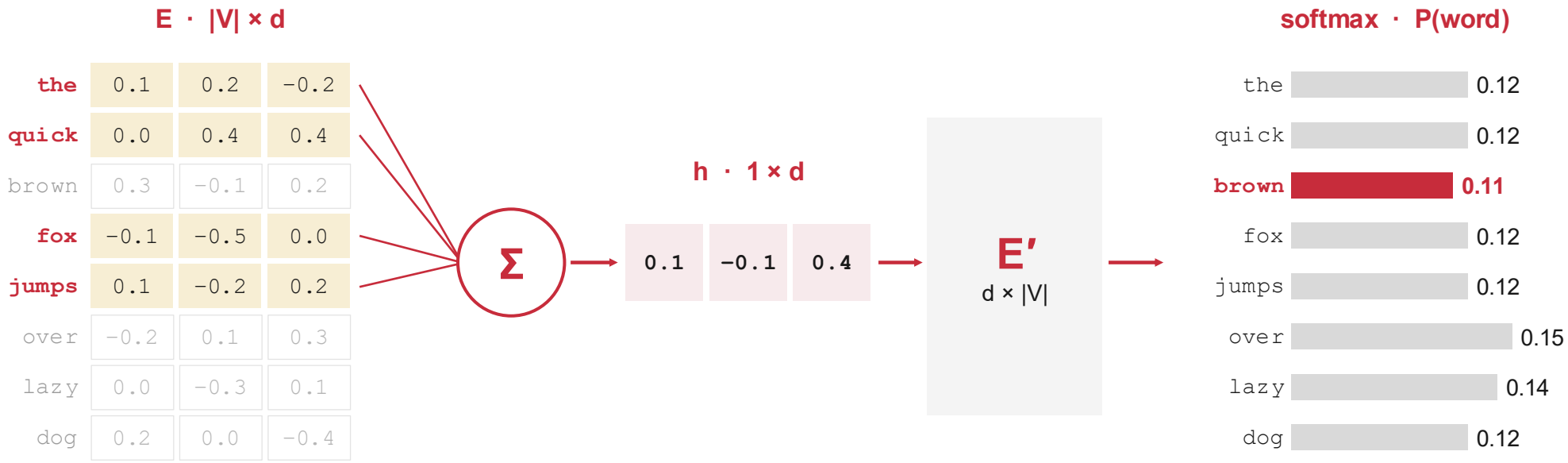
A pretext task. Slide a window; predict the centre word from its neighbours (CBOW) or the neighbours from the centre (SkipGram); nudge the vectors by backpropagation. Keep E, discard the predictor.

WHAT WE GOT

Dense vectors of 50–500 dimensions in which related words sit close. fastText adds subwords, Doc2vec adds whole texts. All static: one vector per word type.

One more look at CBOW. Last week we kept E and discarded the prediction step. Today we reopen that step once, because the part we threw away, the predictor, is exactly the kind of model we study today: a linear classifier.

Inside Word2vec: the hidden layer is a lookup table



INPUT
context words pick their rows

HIDDEN LAYER
lookup, then sum (or average)

OUTPUT LAYER
scores u = h E′

PREDICTION ŷ = softmax(u)
true word: brown

1 · LOSS: CROSS-ENTROPY

$L = -\Sigma_j y_j \log \hat{y}_j$

y is one-hot: 1 for brown, 0 for all others, so one term is left:

$L = -\log \hat{y}_{brown} = -\log 0.11 = 2.17$

2 · ERROR AT THE SCORES u = h E′

$\partial L/\partial u_j = \hat{y}_j - y_j$

softmax + cross-entropy give this clean gradient:

e = [0.12, 0.12, −0.89, 0.12, …]

3 · UPDATE E′ (d × |V|)

$\partial L/\partial E' = h^T e$

every column moves: brown's toward h, the others away

$E' \leftarrow E' - \eta\, h^T e$

4 · ERROR AT THE HIDDEN LAYER

$e_h = e\, E'^T \quad (1 \times d)$

sent to the 4 rows of E that were looked up; all other rows unchanged

row ← row − η e_h

E is what we keep; E′ only served the prediction.

Word2vec's output layer is already a classifier

$$P(\text{word} = j \mid \text{context}) = \text{softmax}(h E')_j$$

multinomial logistic regression, with every vocabulary word as a class

E · |V| × d · BUILDS THE FEATURES

Looked up and summed into h: the input x of the classifier. After training: kept. These rows are the embeddings.

E′ · d × |V| · HOLDS THE WEIGHTS

Column j is the weight vector w for class j (the word j). After training: thrown away.

	WORD2VEC · SESSION 4	TEXT CLASSIFICATION · TODAY
Classes	every word in V: 10^4 to 10^6	2, K, or any subset
Labels come from	the text itself: free	human annotation: expensive
Features x	h, learned with the classifier (from E)	fixed in advance: counts, or rows of E
What we keep	E: the features	w: the weights, like E′

Last week the classifier was a means to get features. This week the features are a means to get a classifier.

Visualizing the embedding space: Discuss Why?

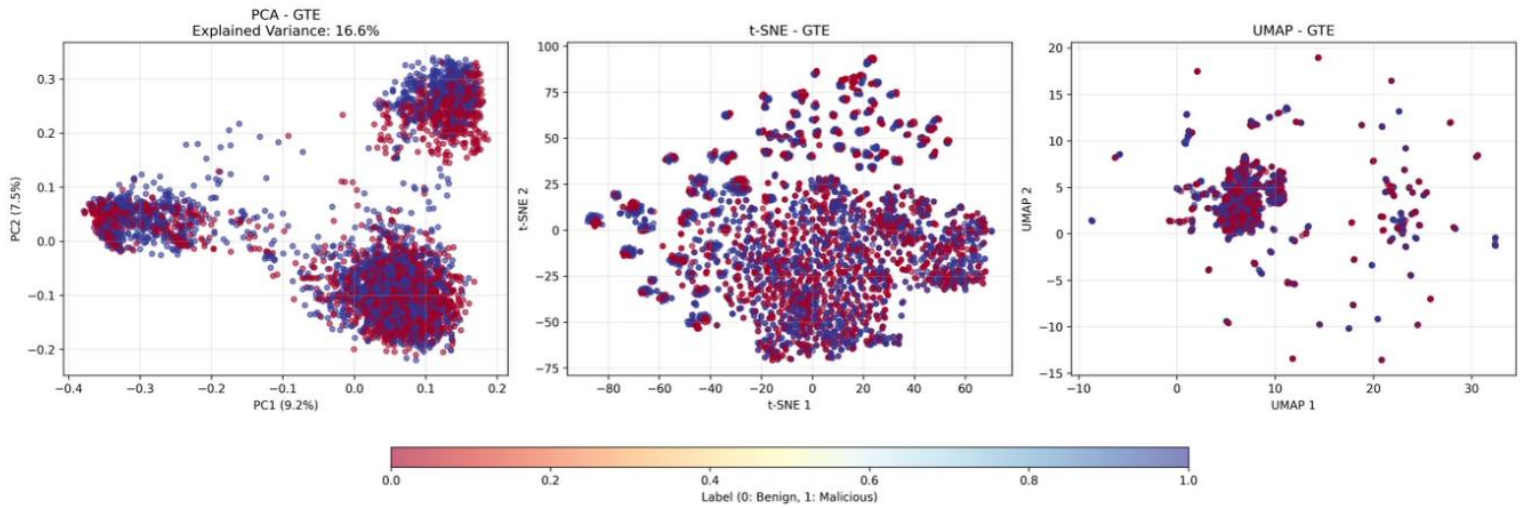
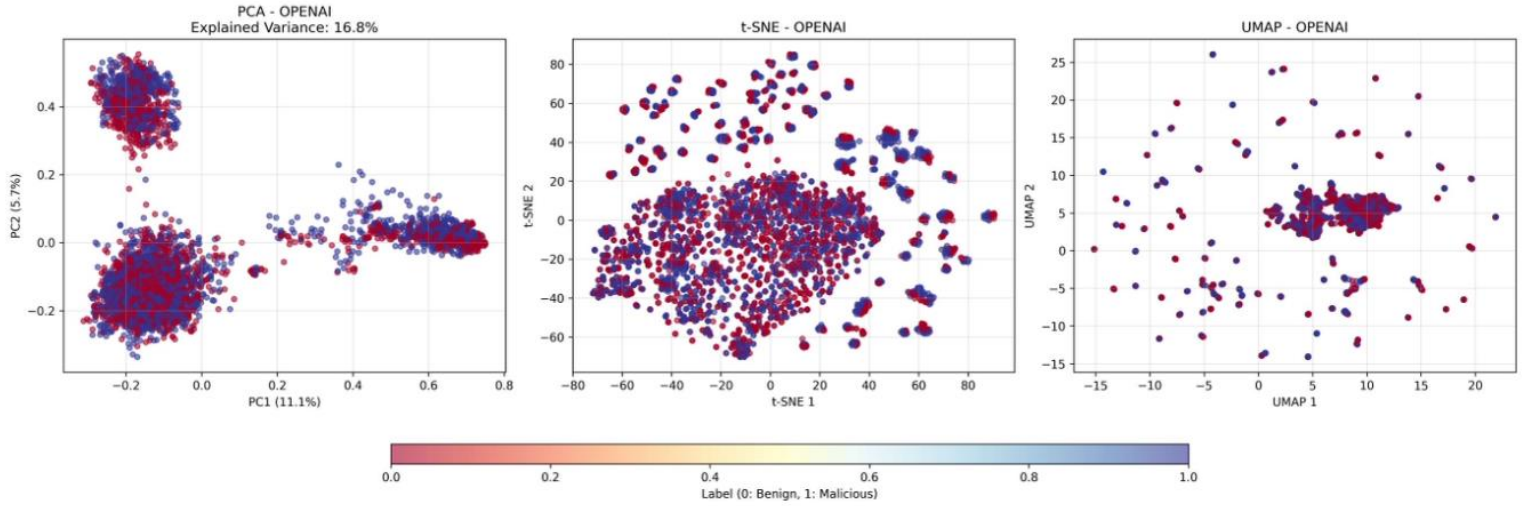


Figure 7. PCA, t-SNE, and UMAP visualisations for GTE-large embeddings. Benign (blue) and malicious (red) samples form overlapping but distinguishable clusters, with UMAP showing clearer separation. This intermediate level of separation aligns with GTE-large's mid-tier performance.

Source: Alamsabi, M.; Tchuindjang, M.; Brohi, S. Embedding-Based Detection of Indirect Prompt Injection Attacks in Large Language Models Using Semantic Context Analysis. *Algorithms* **2026**, *19*, 92. https://doi.org/10.3390/a19010092



Text classification: the task

One question separates the three settings: **how many labels does ONE document get?**

BINARY

2

classes

per document: **exactly one label**

Is this article relevant to the US economy?

MULTICLASS

3+

classes

per document: **still exactly one label**

Is this review negative, neutral or positive?

MULTILABEL

any

number of classes

per document: **any subset: none, one, several, all**

Which desks should file this article?

The number of classes is not what distinguishes them. A multiclass problem may have ten classes, but each document still gets exactly one. Multilabel differs in kind: one document can carry several labels at once.

Why this is the most widely used NLP task

CONTENT CLASSIFICATION AND ORGANIZATION

Tagging large volumes of text to power search, recommendation and routing. Organising email into personal, social and promotions; routing service requests to the right support team.

CUSTOMER SUPPORT

Separating the messages a brand must respond to (actionable) from those it need not (noise). Three tweets can all mention a brand while only one requires a reply.

E-COMMERCE AND SENTIMENT

Classifying reviews as negative, neutral or positive — and increasingly aspect-based, since a single review may praise the food and criticise the service.

AND BEYOND

Language identification, authorship attribution, triaging posts in mental-health support forums, and separating fake news from real news.

Discuss: text classifiers you have already met

1. Think of a system you have used that sorts text: a spam filter, a support-ticket router, content moderation, a CV screener. What were its classes: binary, multiclass or multilabel?

2. Who labelled the examples it learned from, and whose judgement did those labels encode?

3. What does one mistake cost, and who pays for it: a false positive or a false negative?

3 min in pairs → 3 min discussion.

Before you train anything: get a baseline

THE MAJORITY-CLASS BASELINE

80%

Predict the most common label every single time. On a dataset that is 80% one class, this scores 80% accuracy while learning nothing at all.

A RULE-BASED BASELINE

no learning

Lists of positive and negative words compared in the input: lexicon-based sentiment analysis. Fast to build, and it teaches you about the problem.

Report the baseline alongside every result.
An accuracy of 0.81 is excellent against a baseline of 0.55 and worthless against a baseline of 0.80. The comparison is the finding, not the number.

Where do last week's embeddings go? Into x.

The models only weight the numbers in x. Which numbers go into x is our choice, and Sessions 3 and 4 gave us two kinds.

ROUTE A · COUNTS

the running example

"Inflation pushes the interest rate up" → `[0, 1, 0, …, 1, …, 0]`

BoW / TF-IDF: one column per word, ~45,000 columns, almost all zero → **Naive Bayes · logistic regression · SVM**

ROUTE B · EMBEDDINGS

Session 4

"Inflation pushes the interest rate up" → `[0.12, −0.08, …, 0.31]`

look up each word's row of E, then average: 300 columns, all filled → **logistic regression · SVM**

WHY THE RUNNING EXAMPLE USES COUNTS

Text Naive Bayes expects counts

MultinomialNB models how often each word appears in a class. A dense vector needs GaussianNB, a weaker fit.

Readable weights

With counts, every weight belongs to a word you can name. An embedding dimension has no name.

A strong baseline

On a topic task with thousands of labelled articles, counts are hard to beat. Embeddings tend to help when labels are few.

Counting words is a choice of representation, not a limit of the models. What each route buys us comes after the three models.

Session Break

When we return: three classifiers (Naive Bayes, logistic regression, SVMs), each read the same four ways.

Model 1: Naive Bayes

WHAT IT SOLVES	Asks what documents of each class typically look like, then picks the class that most plausibly produced this document.
ASSUMPTION	Features are conditionally independent given the class — the “naive” part. Plainly false for language, and it works anyway.
OBJECTIVE	No iterative training and no loss to minimise. Probabilities are counted from the training data and combined with Bayes’ theorem.
OUTPUT	A probability for each class; the prediction is the class with the maximum.

Family: generative

It models how each class produces text.

Boundary shape

Linear. Multinomial Naive Bayes produces a linear decision rule, just like the next two models.

Cost of the assumption

Confidence is overstated. Trust the ranking, not the number.

Typical use: spam filtering, and a first pass on any high-dimensional sparse count matrix. It trains in seconds, so there is rarely a reason not to run it as your baseline.

Naive Bayes: Bayes' theorem, plus one naive step

$$\underbrace{P(\text{class} \mid x)}_{\text{POSTERIOR}} = \underbrace{P(x \mid \text{class})}_{\text{LIKELIHOOD}} \times \underbrace{P(\text{class})}_{\text{PRIOR}} / \underbrace{P(x)}_{\text{EVIDENCE}}$$

STEP 1 · DROP THE EVIDENCE (how probable is this collection of words?

P(x) is the same for every class, so it cannot change the winner:

$$P(\text{class} \mid x) \propto P(x \mid \text{class}) \times P(\text{class})$$

STEP 2 · THE NAIVE STEP

Treat the features of x as independent given the class:

$$P(x \mid \text{class}) = \prod_j P(x_j \mid \text{class})$$

WHAT A FEATURE x_j IS DECIDES WHICH NAIVE BAYES YOU USE

COUNTS · MultinomialNB

x_j = how often word j occurs. P(word | class) is counted from the training articles of each class.

EMBEDDINGS · GaussianNB

x_j = dimension j: a real number, not a count. One bell curve per dimension and class. It runs, but it is rarely the best choice: use logistic regression or the SVM.

Result: $P(\text{class} \mid x) \propto P(\text{class}) \times \prod_j P(x_j \mid \text{class})$ posterior ∝ prior × likelihood, feature by feature

Discuss: when is a naive assumption good enough?

1. Name two words from your own research texts that almost always appear together. Naive Bayes treats them as independent: what happens to their evidence, and why can the label still be right?

2. Could Naive Bayes take an averaged Word2vec vector as x? Which part of the formula would have to change, and what would you then be assuming?

3 min in pairs → 3 min discussion.

Model 2: Logistic regression

WHAT IT SOLVES	Asks which features distinguish the classes, and learns a weight for each according to how much it moves the decision.
ASSUMPTION	The log-odds of the positive class are linear in the features, so the decision boundary is a hyperplane.
OBJECTIVE	Minimise the logistic loss — equivalently, maximise the likelihood of the labels. Solved by gradient descent.
OUTPUT	A probability p(y = 1 | x), turned into a label by a threshold you control.

Family: discriminative

It learns what separates the classes.

Why it is worth the extra work

- The weights are readable: which terms drove the decision.
- The probability is calibrated enough to threshold.
- Handles imbalance directly with class weights.

Typical use: relevance ranking and risk scoring — anywhere you must report how confident the model is, not just what it decided. The standard MVP in industry and the standard baseline in research.

Why odds, and why the logarithm

Odds: p / (1 − p)

How much more likely the event is than its absence. “3 to 1” means three times as likely to happen as not.

Log-odds: log [p / (1 − p)]

The same information on a scale that runs from −∞ to +∞, centred on zero.

Probability p	Odds p / (1 − p)	Log-odds
0.10	0.11	−2.20
0.25	0.33	−1.10
0.50	1.00	0.00
0.75	3.00	+1.10
0.90	9.00	+2.20

Read the third column. Probability is squashed into [0, 1] and asymmetric — odds run 0 to 1 below an even chance and 1 to ∞ above it. Log-odds are symmetric: 0 is an even chance, and “twice as likely” is the same distance up as “half as likely” is down.

The model assumes $\text{logit}(p) = w^T x + b$. The logit turns a probability into a score; the sigmoid $\sigma(z) = 1 / (1 + e^{-z})$ turns the score back into a probability. They are inverses — the sigmoid does not create the log-odds, it undoes them.

The log-loss, in one rule

loss = −log (probability the model gave to the correct answer)

1 SCORE

from the “one score” slide

$$z = w^T x + b = 1.7$$

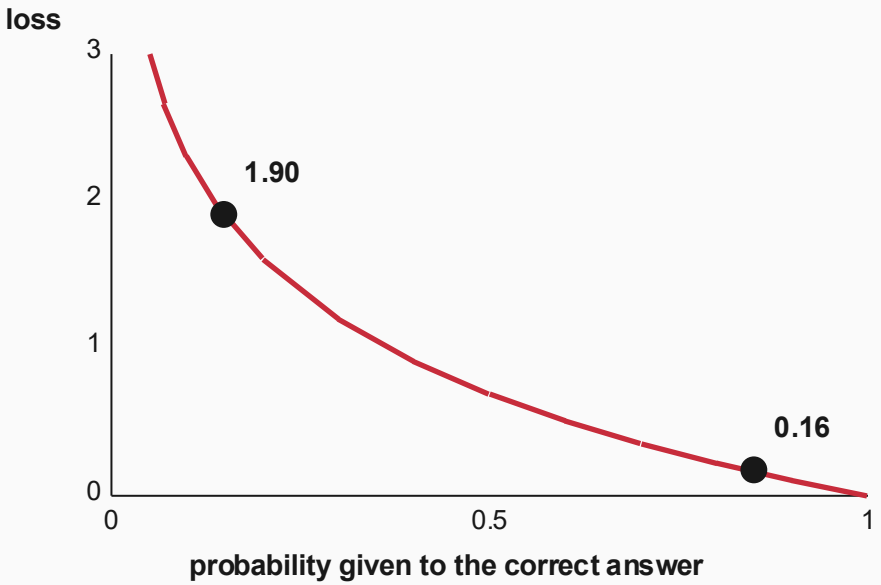
2 PROBABILITY

the model says: 85% relevant

$$p = \sigma(1.7) = 0.85$$

3 COMPARE WITH THE TRUTH

truly relevant	right answer got 0.85	**−log 0.85 = 0.16**
truly not relevant	right answer got 0.15	**−log 0.15 = 1.90**



Training: choose w and b so that the loss, summed over all training articles, is as small as possible.
Textbook form: $L = -[\, y \log p + (1 - y) \log(1 - p) \,]$ is the same rule; y = 1 or y = 0 keeps one term.

Model 3: Support vector machine

WHAT IT SOLVES	Asks for the boundary that sits as far as possible from the nearest examples of either class — the maximum-margin hyperplane.
ASSUMPTION	The classes can be separated with a wide margin, possibly only after mapping the features into a higher-dimensional space.
OBJECTIVE	Maximise that margin while penalising points that fall inside it or on the wrong side. Only the nearest points influence the result.
OUTPUT	Which side of the hyperplane the document falls on. No probability by default — LinearSVC has no predict_proba.

Family: discriminative

Like logistic regression — geometry, not likelihood.

Boundary shape

Linear with a linear kernel; non-linear with an RBF or polynomial kernel. The only model here that can bend.

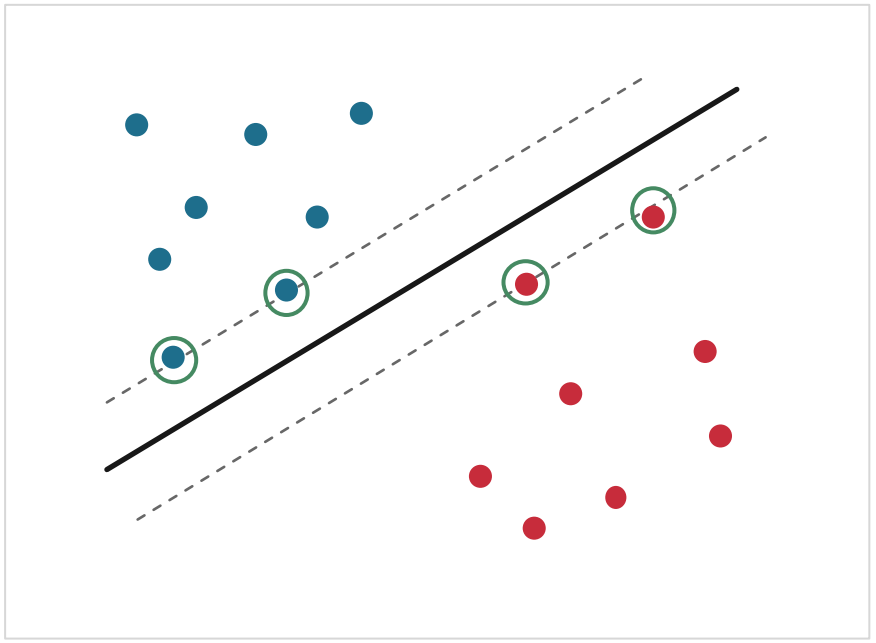
Cost of the geometry

Training time grows quickly with the number of documents.

Typical use: high-dimensional text with moderate sample sizes — the classic strong baseline for document classification before neural methods, and still competitive.

Support vector machines: the margin idea

The widest corridor between the classes



Solid line: the boundary. Dashed lines: the margin. Ringed points: the support vectors that define it.

ONLY THE NEAREST POINTS MATTER

The boundary is fixed by the ringed support vectors alone. Delete every other document and you get the same model, a real difference from logistic regression, where every example contributes.

A WIDER MARGIN GENERALISES BETTER

Leaving room either side means a new document has to be quite unusual before it lands on the wrong side. This is why SVMs hold up well when features vastly outnumber documents.

KERNELS: BENDING THE BOUNDARY

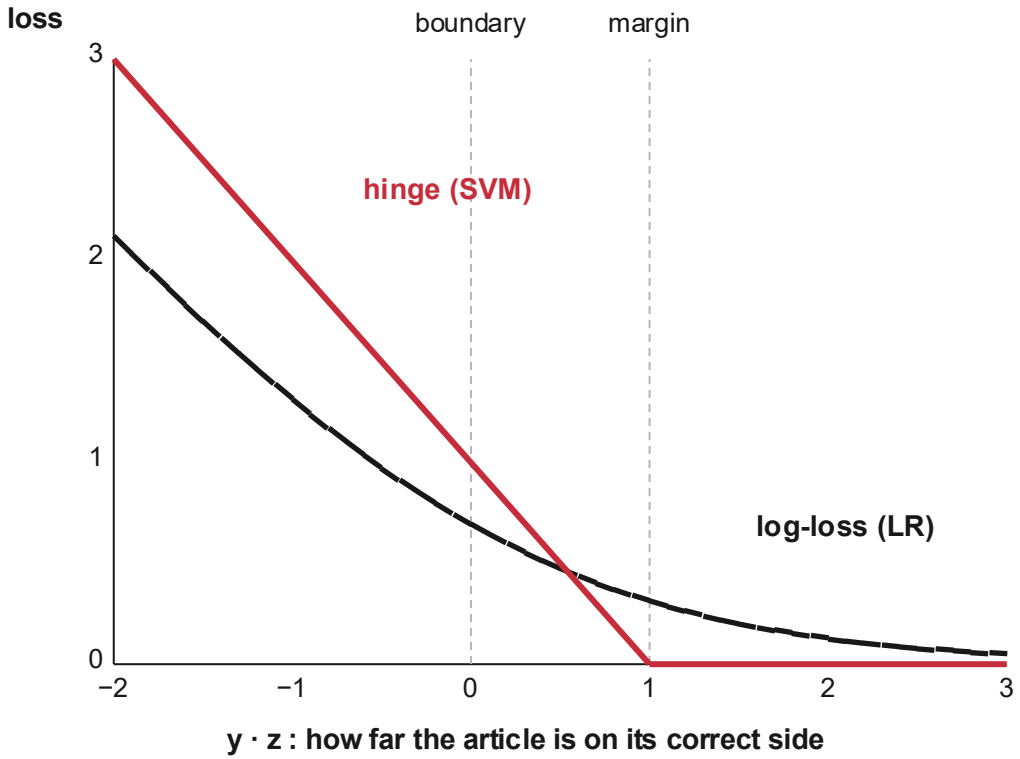
If no straight line separates the classes, map the features into a higher-dimensional space where one does. LinearSVC does not do this; SVC with an RBF or polynomial kernel does — at considerably more training time.

The SVM, intuitively: four ideas, four formulas

1	**THE BOUNDARY** Same score as the other models. The line is where the score is exactly zero.	$w^T x + b = 0$ $z = w^T x + b$, labels $y \in \{-1, +1\}$
2	**THE MARGIN** The closest articles must score ≥ +1 or ≤ −1. The corridor between them is 2 / ‖w‖ wide: smaller weights, wider corridor.	$y_i (w^T x_i + b) \geq 1$ corridor width = $2 / \|w\|$
3	**WIDEST CORRIDOR = SMALLEST WEIGHTS** Maximising the width is the same as minimising the size of w, while every article stays on its side.	$\min \frac{1}{2} \|w\|^2$ subject to $y_i (w^T x_i + b) \geq 1$
4	**REAL DATA OVERLAP** Let articles enter the corridor or cross the line, but charge a fee for each: the hinge loss. C is the price.	$\min \frac{1}{2} \|w\|^2 + C \sum_i \max(0, 1 - y_i z_i)$ $yz \geq 1$: fee 0 · $yz = 0.4$: fee 0.6

An SVM is the widest corridor plus a fee for the articles that cross into it.
Articles safely outside pay nothing and do not move the line; only those on or inside the corridor, the support vectors, shape it.

Logistic regression vs SVM: the same line, a different loss



LOGISTIC REGRESSION · LOG-LOSS

The loss never reaches zero, so every article keeps pulling on the line, even well-classified ones. Output: a probability.

SVM · HINGE LOSS

Zero loss once an article is past the margin. Only the articles near the line, the support vectors, decide where it sits. Output: a side, no probability. Can bend with a kernel.

IN PRACTICE ON TEXT

Usually similar accuracy. Logistic regression if you need probabilities; SVM for many features when only the label matters.

Why regularization: with thousands of word columns and few articles, a weight on a rare word can grow without limit. Both models penalise large weights; C in scikit-learn (default 1.0) sets the strength, and smaller C means a stronger penalty. Tune it by cross-validation.

What these models can represent: curved boundaries and label settings

Can it model a non-linear relationship?

Naive Bayes	**No**	Multinomial NB gives a linear decision rule. The probabilistic framing does not make it curved.
Logistic regression	**No, not by itself**	The boundary is a hyperplane. Non-linear effects must be engineered in as features — interactions, polynomial terms, binned variables.
SVM	**Yes, with a kernel**	LinearSVC is linear. SVC with an RBF or polynomial kernel learns curved boundaries.

Binary, multiclass, multilabel?

All three handle binary and multiclass. Naive Bayes is natively multiclass; logistic regression uses multinomial (softmax) or one-vs-rest; SVMs use one-vs-rest. None is natively multilabel — for that you wrap the estimator in OneVsRestClassifier, which fits one independent binary model per label and lets a document receive any subset.

Sparse or dense x: what do embeddings buy a linear model?

WORD-BASED · COUNTS

$$z = \Sigma\ w_word + b$$

sum over the words in the article

One free weight per word, learned only from the labelled articles.

EMBEDDING-BASED · AVERAGED WORD2VEC

$$z = mean(\ w \cdot v_word\ ) + b$$

the same as w · mean(v_word): still one term per word

A word's score is w · v_word: fixed by where the word sits in the embedding space.

	COUNTS	AVERAGED EMBEDDINGS
"superb" never appears in the labelled data	contributes nothing	scored like "excellent", its neighbour
One decisive word in a long article	keeps its full weight	diluted by the average of all other words
"not good": order and context	lost: "not" + "good"	lost: "not" + "good"

What we earn is similarity between words, learned from unlabelled text, not context.
It pays when labelled data is scarce and wording varies. With thousands of labels and a keyword-driven task, counts usually match it and keep readable weights. Reading context needs a model of the sequence: Sessions 8–9.

Reading the result: which number should you report?

ACTUAL	PREDICTED: relevant	PREDICTED: not relevant
relevant	**TP** correctly found	**FN** missed
not relevant	**FP** false alarm	**TN** correctly ignored

ACCURACY	(TP + TN) / everything. How often the model is right — and useless here, since always predicting “not relevant” scores 80%.
PRECISION	TP / (TP + FP). Of the articles we flagged, how many really were relevant?
RECALL	TP / (TP + FN). Of the articles that really were relevant, how many did we find? This is the number we compare across the three models next.
F1	Precision and recall in one number; a harmonic mean, not an average, so it is pulled down by whichever is weaker. Precision 0.90 with recall 0.10 averages 0.50, but gives F1 = 0.18.
ROC AUC	Take one relevant and one irrelevant article at random: AUC is the chance the model ranks the relevant one higher. 0.5 is a coin flip. It judges the ordering, not one threshold — so it needs scores, which LinearSVC withholds.

Precision and recall trade off. Flag everything and recall hits 1.0 while precision collapses; flag almost nothing and the reverse. Which you favour is a decision about costs, not about statistics.

Discuss: when is a classifier good enough?

1. Your model flags economy-related articles for a ministry's morning briefing. Which mistake is worse: missing a relevant article, or including an irrelevant one? Which metric follows from your answer?

2. Accuracy is 85% and the majority baseline is 80%. What else would you want to see before calling the model a success?

3. The model was trained on news from ten years ago. Would you trust its scores on articles published today? What might have changed?

3 min in pairs → 3 min discussion.

When the classifier does poorly, work the list

01	**Too many features**	Extracting everything gives a large sparse vector where most features are rare noise. Cap the vocabulary.
02	**Class imbalance**	The learning process skews toward the majority class. One of the most common reasons a classifier fails — always check first.
03	**The wrong algorithm**	No algorithm is best on all data. Try several and compare on the same split.
04	**Weak preprocessing or features**	Revisit Sessions 2–4: normalisation choices and the representation itself both bound what the model can learn.
05	**Untuned parameters**	Hyperparameters and regularisation strength are rarely right by default.

Feature engineering for a linear model

N-GRAMS

Unigrams cannot represent negation. Bigrams let 'not good' exist as its own feature rather than as 'not' plus 'good'.

TF–IDF vs RAW COUNTS

Down-weights terms common to every document. sublinear_tf dampens long documents dominating on repetition alone.

VOCABULARY LIMITS

min_df, max_df, max_features. Cutting 45,000 features to 5,000 lowered accuracy but raised minority-class recall from 0.42 to 0.65.

SCALING AND NORMALISATION

Neither model is scale-invariant once regularized, the penalty treats all weights alike. L2-normalise rows for text.

INTERACTIONS BY HAND

The model cannot discover that two features matter only together. If an interaction matters, construct the column yourself.

FEATURES BEYOND THE TEXT

Document length, source, section, publication date. Often the cheapest accuracy in the whole pipeline.

The model weights the representation it is given, and can recover nothing that representation discarded.

This is the last session in which the features are entirely your decision. From Session 7 the network learns them — which removes this work, and with it the ability to inspect and justify what the model is looking at.

Where we are in the NLP pipeline



Data acquisition	Text extraction, normalization	Representation / features	Basic models	Neural models	Evaluation	Final evaluation + XAI	Orchestration
Not covered	Session 2	**Sessions 3–4**	SESSION 5 & 6	Sessions 7–9	Throughout	Session 10	Sessions 11-12

scikit-learn imblearn · next: topic models (Session 6), then neural networks (Session 7)

Today: train and evaluate a supervised classifier on those features. Next: LDA and BERTopic, which need no labels at all.

Appendix

For reference: the log-loss from maximum likelihood, its gradient, the SVM derived, and kernels.

Where the logistic loss comes from

STEP 1 · ONE DOCUMENT

$$p(y \mid x) = p^{y}\,(1-p)^{1-y} \qquad \text{with} \quad p = \sigma(w^{\top}x + b)$$

How likely was the label we actually saw? Because y is either 0 or 1, one exponent switches one factor off: y = 1 leaves p, y = 0 leaves 1 − p.

↓ **multiply over every document, assuming they are independent**

STEP 2 · THE WHOLE TRAINING SET — the likelihood

$$\mathcal{L}(w, b) = \prod_i p_i^{y_i}\,(1-p_i)^{(1-y_i)}$$

Still a probability, not a loss: how likely is this entire set of labels, under these weights? We want this number to be LARGE.

↓ **take logarithms, then flip the sign**

STEP 3 · THE LOSS — one number to minimise

$$L(w, b) = -\Sigma_i \left[\, y_i \log \sigma(z_i) + (1-y_i) \log(1-\sigma(z_i)) \,\right]$$

The log turns the product into a sum — easier to differentiate, and it stops thousands of factors below 1 from underflowing. The minus sign turns "make it large" into "make it small", which is what an optimiser does.

Only the first line is a choice. Once you accept $p = \sigma(w^{\top}x + b)$, steps 2 and 3 are algebra. That is why the logistic loss is not one option among several — it is what maximum likelihood gives you.

The gradient, and why the update rule is so simple

$$\partial L/\partial w_j = (\partial L/\partial a) \cdot (da/dz) \cdot (\partial z/\partial w_j) \qquad \text{where } a = \sigma(z)$$

LOSS w.r.t. ACTIVATION

$\partial L/\partial a = (a - y) / [\, a(1 - a) \,]$

Differentiating the log-loss.

SIGMOID w.r.t. NET INPUT

$da/dz = a(1 - a)$

The sigmoid's derivative, written in terms of itself.

NET INPUT w.r.t. WEIGHT

$\partial z/\partial w_j = x_j$

z is linear in w, so this is just the feature value.

MULTIPLY OUT

$$\partial L/\partial w_j = (a - y) \cdot x_j \qquad \partial L/\partial b = (a - y)$$

SO THE UPDATE IS

$$w_j := w_j + \eta\,(y - a)\,x_j \qquad b := b + \eta\,(y - a)$$

Error × feature. The a(1 − a) terms cancel, leaving the prediction error scaled by how strongly the feature was present. A word absent from a document ($x_j = 0$) never moves its weight.

The loss is convex, so there is a single global minimum — which is why lbfgs, newton-cg and saga all reach the same answer, and why nothing here needs a random restart.

The SVM, derived

01 FIX THE SCALE

$y_i\,(w^T x_i + b) \geq 1$ for every document i

w and b can be rescaled freely without moving the boundary, so we choose the scale that puts the closest points at exactly 1.

02 MEASURE THE CORRIDOR

margin width $= 2 / \|w\|$

The distance between the two dashed hyperplanes. A wider corridor means a smaller weight vector — that is the whole link.

03 TURN IT INTO A MINIMISATION

$\min$ over w, b: $\tfrac{1}{2}\|w\|^2$ subject to $y_i\,(w^T x_i + b) \geq 1$

Maximising $2/\|w\|$ is minimising $\|w\|$. The square and the ½ are there only to make the derivative clean.

04 LET SOME POINTS MISBEHAVE

$\min\ \tfrac{1}{2}\|w\|^2 + C\,\Sigma_i\,\xi_i,\quad y_i(w^T x_i + b) \geq 1 - \xi_i,\quad \xi_i \geq 0$

Real classes overlap, so the hard constraint has no solution. Each slack ξ_i buys one violation, and C is the price.

05 READ OFF THE LOSS

$\xi_i = \max(0,\, 1 - y_i\, z_i)$ → the hinge loss

At the optimum each slack takes its smallest permitted value. So the soft-margin SVM is hinge loss plus an L2 penalty.

The same shape as logistic regression. A loss over the documents plus a penalty on $\|w\|$. Only the loss differs: the hinge is flat once a point is correct by a margin, while the logistic loss never quite reaches zero.

The dual, and where kernels come from

$$\max_{a} \; \Sigma_i \alpha_i - \tfrac{1}{2} \Sigma_i \Sigma_j \alpha_i \alpha_j y_i y_j \langle x_i, x_j \rangle$$

subject to $0 \le \alpha_i \le C$ and $\Sigma_i \alpha_i y_i = 0$ — one variable α_i per document, not per feature

WHERE THE SUPPORT VECTORS COME FROM

$w = \Sigma_i \alpha_i y_i x_i$

Most α_i are exactly zero. Only documents with $\alpha_i > 0$ enter the weight vector — these are the support vectors, and this equation is why deleting the rest changes nothing.

WHERE KERNELS COME FROM

$\langle x_i, x_j \rangle \rightarrow K(x_i, x_j) = \langle \varphi(x_i), \varphi(x_j) \rangle$

The features appear only inside an inner product. Swap it for a kernel and the model works in a richer space without ever computing $\varphi(x)$. That substitution is the kernel trick.

KERNEL	K(x, x′)	USE IT WHEN
Linear	$\langle x, x' \rangle$	**high-dimensional sparse text — almost always**
Polynomial	$(\gamma\langle x, x' \rangle + r)^d$	feature interactions matter and d is small
RBF	$\exp(-\gamma\|x - x'\|^2)$	dense low-dimensional features, curved boundary

Which solver you get is a library decision. LinearSVC solves the primal and scales to many documents but stays linear; SVC solves the dual and accepts kernels, at roughly n^2 to n^3 cost. That is the real reason the lab caps the SVM at 1,000 features.

Source: Vajjala et al. (2020), Chapter 4, 'Support Vector Machine'; scikit-learn user guide, §1.4.